

Table Name	Column	Data type	Sample value	Data Problems	Preprocessing solutions (if any)	New derived columns (if any)
Identifier	client_ID	Text	CUST_00001	None. Verify uniqueness: 32,581 unique IDs = total rows → OK	SQL: SELECT COUNT(DISTINCT client_ID), COUNT(*) FROM credit_risk → must match. Keep as primary key only, exclude from modeling.	None
Demographics	person_age	Int	min=20, median=26, max=144	5 rows with age > 100 (max=144) — biologically impossible. Possible data entry errors.	SQL: DELETE FROM credit_risk WHERE person_age > 100; (5 rows ≈ 0.015%, safe to remove). Alternative: cap at 80 if want to preserve rows.	age_bucket (20-25, 26-30, 31-40, 41-55, 56+) for WoE/IV and segment analysis
	person_income	Int	min=4,000, median=55,000, max=6,000,000	Extreme outliers — 53 rows with income > 500K (max=6M). Right-skewed distribution will distort means and break linear models.	SQL: Winsorize at 99th percentile (cap above P99 to P99 value). Alternatively, Python fallback for log1p transform if needed for modeling. Document choice in notebook.	log_income (for modeling); income_bucket (Low <30K / Mid 30-75K / Upper 75-150K / High >150K)
	person_home_ownership	Text	{'RENT', 'OWN', 'MORTGAGE', 'OTHER'}	OTHER category meaning unclear (e.g., living with family? government housing?). Could mask risk signal.	SQL: SELECT person_home_ownership, COUNT(*), AVG(loan_status) FROM credit_risk GROUP BY 1 → if OTHER very small (<1%), consider merging with RENT; otherwise keep as-is and document interpretation.	is_homeowner flag (OWN/MORTGAGE = 1, RENT/OTHER = 0) for simpler binary feature
	person_emp_length	Float	min=0, median=4, max=123	(1) 895 missing values (2.75%). (2) max=123 years — biologically impossible. (3) Need to check if missingness correlates with employment_type=Unemployed.	SQL: Step 1 — cap at 50 (UPDATE ... SET person_emp_length=50 WHERE person_emp_length>50). Step 2 — for missing: if employment_type=Unemployed → impute 0; else impute median by employment_type group. Python fallback if SQL imputation gets complex.	emp_length_bucket (0-2 / 3-5 / 6-10 / 10+) for binning analysis
	gender	Text	{'Male', 'Female'}	⚠ PROTECTED VARIABLE (ECOA / fair lending). Cannot be used as model input regardless of statistical signal.	SQL: Keep column for descriptive EDA only (distribution check, default rate by gender for fairness audit). FLAG for exclusion from any predictive model. Document in Approaches section.	None — do not engineer features from this column
	marital_status	Text	{'Single', 'Married', 'Divorced', 'Widowed'}	⚠ PROTECTED VARIABLE (ECOA). Same restriction as gender — cannot use for credit decisioning.	SQL: Descriptive EDA only. FLAG for exclusion from modeling. Document rationale.	None
	education_level	Text	{'High School', 'Bachelor', 'Master', 'PhD'}	⚠ PROTECTED VARIABLE under fair lending (proxy for socioeconomic status). Exclude from scoring.	SQL: Descriptive EDA only. FLAG for exclusion from modeling.	None

Table Name	Column	Data type	Sample value	Data Problems	Preprocessing solutions (if any)	New derived columns (if any)
	employment_type	Text	{'Full-time', 'Part-time', 'Self-employed', 'Unemployed'}	Suspicious: in synthetic data, Unemployed default rate may be unrealistically close to Full-time (vs reality). Flag as potential synthetic artifact (H12 hypothesis).	SQL: Compute default rate by employment_type. If distributions look too uniform, document as data limitation in Limitations section. Keep as candidate feature.	is_stable_employment flag (Full-time/Self-employed = 1, else 0)
Loan Characteristics	loan_intent	Text	{'PERSONAL','EDUCATION','MEDICAL','VENTURE','HOMEIMPROVEMENT','DEBTCONSOLIDATION'}	DEBTCONSOLIDATION typically has highest default rate, BUT this may be confounded with high DTI / poor credit history (H3 hypothesis). Cannot conclude intent drives risk without controlling for confounders.	SQL: Keep as categorical feature. Compute default rate by intent AND by intent × DTI bucket to check confounding before drawing conclusions.	is_consumption_loan (PERSONAL/SHOPPING/DEBTCONSOLIDATION) vs is_investment_loan (EDUCATION/VENTURE/HOMEIMPROVEMENT)
	loan_grade	Text	{'A','B','C','D','E','F','G'} — 7 grades	⚠️ OUTPUT of Nova Bank's current scoring system, NOT raw input. Using it as model feature = circular logic (predicting score from score). Strong correlation with default by construction.	SQL: Keep column for VALIDATION purposes only (does current grading align with actual default behavior?). FLAG exclusion from new scoring model. Use to compare new model vs existing system.	grade_numeric (A=1, B=2, ..., G=7) for ordinal analysis only; is_subprime (D-G) for downstream policy simulation
	loan_amnt	Int	min=500, median=8,000, max=35,000	No obvious data quality issue. Range plausible for personal loans. Right-skewed distribution.	SQL: No cleaning needed. Verify no negative or zero values. Consider log transform for modeling if used as raw feature.	loan_amnt_bucket (Small <5K / Medium 5-15K / Large 15-25K / X-Large 25K+)
	loan_int_rate	Float	min=5.42, median=10.99, max=23.22 (%)	3,116 missing values (9.56%) — significant gap. Pattern not random: likely missing for specific grades or origination channels. Must investigate before imputing.	SQL Step 1: SELECT loan_grade, COUNT(*), AVG(loan_int_rate), COUNT(loan_int_rate)/COUNT(*) AS coverage FROM credit_risk GROUP BY 1 → check missingness pattern by grade. Step 2: Median impute by loan_grade (preserves the rate-grade relationship). If pattern not by grade, escalate to Python for MICE/iterative imputation.	int_rate_bucket; rate_vs_grade_avg (deviation from grade median, for pricing alignment H10 analysis)
	loan_term_months	Int	{12, 24, 36, 60}	Only 4 discrete values — effectively categorical despite int type.	SQL: Treat as categorical in modeling (one-hot or ordinal). No cleaning needed.	is_long_term flag (>=36 months); term_years (for interpretability)
	loan_percent_income	Float	min=0.0, median=0.15, max=0.83	⚠️ Likely REDUNDANT with loan_to_income_ratio (expected correlation ≈ 1.0). Need to verify and drop one to avoid multicollinearity.	SQL: SELECT CORR(loan_percent_income, loan_to_income_ratio) FROM credit_risk → if > 0.99, drop loan_to_income_ratio (loan_percent_income has cleaner [0,1] range and 77 unique values vs 9,914). Document decision.	lpi_bucket (5 equal-width or quantile bins) for WoE/IV; LPI > 0.3 flag (find threshold from data first per anti-bias rule)

Table Name	Column	Data type	Sample value	Data Problems	Preprocessing solutions (if any)	New derived columns (if any)
Risk Ratios	loan_to_income_ratio	Float	min=0.0008, median=0.148, max=0.83	Redundant with loan_percent_income (see above). Same business meaning, slightly different precision.	SQL: Drop after verification (ALTER TABLE ... DROP COLUMN loan_to_income_ratio) OR exclude from feature set.	None — will be dropped
	other_debt	Float	min=225, median=8,995, max=1,187,999	Extreme right-skew (max ≈ 5,300× the min). All 32,581 unique values → continuous, no caps. Needs transform for modeling.	SQL: Verify no negative values. Winsorize at 99th percentile. For modeling, log1p transform (Python fallback if needed).	log_other_debt; debt_per_income = other_debt / person_income
	debt_to_income_ratio	Float	min=0.065, median=0.333, max=1.054	max > 1.0 (DTI > 100%) — possible but rare; ~few rows have debt exceeding income. Verify whether these are data errors or legitimate over-leveraged cases.	SQL: SELECT COUNT(*) FROM credit_risk WHERE debt_to_income_ratio > 1.0 → if very few (<50), inspect manually. Otherwise keep, as over-leveraged borrowers are exactly the high-risk segment of interest.	dti_bucket (5 quantile bins); dti_above_threshold flag (threshold from data first via H6, benchmark against 0.43 QM rule after)
	open_accounts	Int	min=0, median=8, max=15	No obvious issue. Discrete integer, reasonable range.	SQL: No cleaning needed.	open_accounts_bucket (None=0 / Few 1-3 / Moderate 4-8 / Many 9+) — note this column could also live in Credit Bureau group
	credit_utilization_ratio	Float	min=0.05, median=0.50, max=0.95	⚠ SUSPICIOUSLY UNIFORM distribution (0.05 to 0.95, median ≈ 0.50). In real data this is a top FICO predictor — but here correlation with default ≈ 0. Likely synthetic artifact (H12 hypothesis).	SQL: Do NOT drop yet (anti-bias rule #3). First: check WoE/IV after binning, check multicollinearity with loan_grade (which may have absorbed the signal), check interaction effects. Then conclude in Limitations section.	cu_bucket (Low <0.3 / Mid 0.3-0.7 / High >0.7) for WoE/IV testing
Credit Bureau	cb_person_default_on_file	Text	{'Y', 'N'}	No data quality issue. Strong predictor expected (past behavior predicts future).	SQL: Convert to binary 0/1 for modeling (CASE WHEN cb_person_default_on_file="Y" THEN 1 ELSE 0 END).	has_prior_default (binary 0/1)
	cb_person_cred_hist_length	Int	min=2, median=4, max=30	Verify logical relationship: cred_hist_length should not exceed (person_age - 18). Edge case check needed.	SQL: SELECT COUNT(*) FROM credit_risk WHERE cb_person_cred_hist_length > (person_age - 18) → if any inconsistencies, flag rows for review.	thin_file flag (cred_hist < 3 years) — important for "thin file" borrower segmentation
	past_delinquencies	Int	min=0, median=0, max=6	⚠ SUSPICIOUSLY LOW correlation with default (~0). In reality this is a top 3 predictor. Likely synthetic artifact (H12 hypothesis). Heavy zero-inflation.	SQL: Same as credit_utilization_ratio — do NOT drop. Check WoE/IV, multicollinearity, interaction. Flag in Limitations if confirmed non-predictive.	has_any_delinquency flag (0 vs ≥1); delinquency_bucket (0 / 1-2 / 3+)

Table Name	Column	Data type	Sample value	Data Problems	Preprocessing solutions (if any)	New derived columns (if any)
Geographic	country	Text	{'USA', 'UK', 'Canada'}	Distribution may be artificially uniform (synthetic data). Verify share by country.	SQL: SELECT country, COUNT(*), AVG(loan_status) FROM credit_risk GROUP BY 1 → check balance and default rate differences.	None directly; may interact with city/state
	state	Text	9 states across USA/UK/Canada (e.g., 'California', 'Ontario', 'Wales')	Mixes US states, Canadian provinces, UK constituent countries. Cardinality low (9) so manageable.	SQL: Keep as-is for descriptive geographic analysis. For modeling, target-encode or one-hot.	None
	city	Text	18 cities (e.g., 'Toronto', 'London', 'New York City')	High-cardinality but manageable (18). Likely uniformly distributed (synthetic).	SQL: For modeling, target-encode (mean default rate per city) or aggregate to state. Keep raw for Power BI map visualization.	city_default_rate (target-encoded mean — careful with leakage; compute on train only)
	city_latitude	Float	min=29.76, median=46.81, max=55.95	Only 18 unique values — fully derived from city. Redundant if city is used.	SQL: Keep for Power BI map visualization only. Do NOT use as a model feature (collinear with city).	None
	city_longitude	Float	min=-123.37, median=-75.70, max=-0.13	Same as latitude — 18 unique values, derived from city.	SQL: Visualization only.	None
Target	loan_status	Int	{0 = Non-default (78.18%), 1 = Default (21.82%)}	Mild class imbalance (78/22). Not severe enough for SMOTE. Use class_weight='balanced' in sklearn if modeling.	SQL: No cleaning. Verify: SELECT loan_status, COUNT(*) FROM credit_risk GROUP BY 1 → confirm only 0/1 values present.	None — this is the target

Topic	Description
Dataset	Credit_Risk_Dataset.xlsx — 32,581 rows × 29 columns. Synthetic data simulating Nova Bank personal loans across USA, UK, Canada.
Target variable	loan_status: 0=Non-default (78.18%), 1=Default (21.82%). Class imbalance is mild → use class_weight='balanced', NOT SMOTE.
Tool chain	SQL first (Azure SQL Edge on Docker, VS Code) for data cleaning → Python (pandas/sklearn) for EDA & modeling → Power BI for final dashboard.
Protected variables	gender, education_level, marital_status — must NOT be used in scoring model (ECOA / fair lending). Keep for descriptive EDA and fairness audit only.
Circular logic	loan_grade is OUTPUT of existing scoring system. Using it as a new model input = circular. Use only to VALIDATE the new model vs current grading.
Threshold finding	Find thresholds (DTI cutoff, LPI cutoff, etc.) FROM DATA FIRST using WoE/IV or default rate by bucket. Only then benchmark with regulation (e.g., 0.43 QM Rule).
Suspect synthetic artifacts	past_delinquencies and credit_utilization_ratio have suspiciously low correlation with target (~0). In reality top predictors. Do NOT dismiss as useless — first check multicollinearity, WoE/IV, and interactions. Then document as limitation.
Redundant columns	loan_percent_income ≈ loan_to_income_ratio (expected correlation ≈ 1.0). Verify with SQL CORR(), keep loan_percent_income (cleaner [0,1] range), drop loan_to_income_ratio.
Data quality issues to fix	(1) person_age > 100 — 5 rows, remove. (2) person_emp_length missing 895 + max=123 — impute by employment_type + cap at 50. (3) person_income > 500K — 53 outliers, Winsorize at P99. (4) loan_int_rate missing 9.56% — median-impute by loan_grade.
Derived columns strategy	All buckets/bins to be created in SQL where possible (CASE WHEN). Use for WoE/IV in Python. Document each derived column rationale in Approaches section.

Cleaning workflow	Step 1: Load Excel → Azure SQL Edge (Docker) via mssql-tools / Python pyodbc bulk insert. Step 2: SQL queries for outlier removal, redundancy drops, imputation, derived columns. Step 3: Verify cleaned table → export for Python EDA. Step 4: Document all decisions in notebook with WHY for each transformation.
Issue tree linkage	Each preprocessing decision maps to a hypothesis (H1-H14) from the Issue Tree. E.g., dropping loan_to_income_ratio enables clean H5/H7 testing; capping age enables clean H1 testing.